Tu trabajo dentro del proyecto

Avilix, tu responsabilidad: convertir fuentes diferentes a un lenguaje común y conectar todos los pasos del ETL.

¿Qué construimos entre los tres?

Un pipeline ETL es una secuencia de pasos para extraer datos, transformarlos y cargarlos en una base. Aquí recibe registros de cajeros ATM y transferencias digitales. Un registro es una fila u objeto que describe una operación; un lote es el conjunto que procesamos en una ejecución.

El sistema mide problemas, aplica reglas, guarda aceptadas en **transacciones_limpias** y conserva inválidas en **cuarentena_errores**. Omitir un duplicado significa registrar que ya había una copia, no perderlo sin explicación.

Rol	Qué entrega al equipo
Maki / ingeniería 1	Diagnóstico, faltantes, duplicados e IQR.
Avilix / ingeniería 2	Lectura, integración, divisas, catálogo, MinMax y conexión del flujo.
Leo / ingeniería 3	Modelos Pydantic, errores detallados, persistencia y pruebas.

Archivos que debes reconocer

src/lectura.py: entrada de archivos.
src/integracion.py: normalización, divisas y catálogo.
src/transformacion.py: MinMax y segmentos.
src/pipeline.py y src/cli.py: ejecución del flujo.

> Esta guía explica el ETL obligatorio de los puntos 4 y 6 del profesor. No necesitas explicar dashboard ni simuladores. Los módulos se integraron y adaptaron en conjunto: el rol indica responsabilidad técnica, no autoría exclusiva de cada línea.

1. Unir fuentes diferentes

Los archivos describen operaciones con nombres, formatos y campos distintos. Integrar significa definir una representación común para poder aplicar reglas y guardar resultados comparables. No basta con pegar una tabla debajo de otra.

Concepto	ATM	DIGITAL
Identidad	ID de transacción	ID de transferencia
Cuenta	id_cuenta	account_id
Fecha	Fecha / hora del cajero	timestamp: segundos o fecha con zona
Importe	Monto del cajero	amount
Divisa de origen	Se asume MXN	currency: MXN o USD
Operación	Retiro, depósito, pago o consulta	Transferencia digital

La lectura respeta el formato de cada archivo: XLSX para el ATM entregado, JSON para digitales y CSV para el catálogo. La fuente ATM canónica del proyecto está en datasets/transacciones_atm.xlsx; no se deben mezclar copias distintas de la misma fuente.

El orden real del proceso

1. Leer y perfilar los datos crudos.
2. Separar copias exactas.
3. Validar y normalizar formatos y divisas.
4. Detectar conflictos de identidad y registros ya procesados.
5. Cruzar cuentas con el catálogo.
6. Imputar faltantes, aplicar IQR y redondear.
7. Escalar, segmentar y medir el resultado.
8. Guardar aceptadas, cuarentena, omitidas y evidencia.

> Tu código coordina las partes de Maki y Leo. Los formatos inválidos y las cuentas huérfanas se separan antes de estimar estadísticas, para que no alteren medianas y límites.

2. Qué hicimos con las divisas

Se conserva la moneda original y se convierte el importe a MXN para compararlo. La tasa fija del ejercicio es **1 USD = 19.50 MXN**. No se consulta una cotización actual: es una regla reproducible del proyecto.

Campo de SPEI-2026-00003	Valor guardado
original_json.currency / moneda_original	USD / USD
monto_original	28,312.37
monto_mxn	552,091.22
monto_analitico_mxn	69,304.92
es_outlier	true
monto_normalizado / segmento	1.0 / ALTO

```
28,312.37 USD × 19.50 = 552,091.215 MXN
Redondeo a centavos HALF_UP = 552,091.22 MXN
```

Se utiliza **Decimal** para calcular dinero con precisión decimal y redondeo explícito. HALF_UP hace que un medio centavo se redondee hacia arriba en estos valores positivos. Para MXN, el factor es 1.

Tres importes, tres significados

Original: lo recibido en la fuente, en su moneda.
MXN: importe de operación convertido; conserva su significado monetario.
Analítico MXN: versión limitada por IQR para cálculos estadísticos. No representa una operación de menor valor.

> No se dejó todo en USD. Tampoco se borró la moneda original. Estos campos están dentro del JSON de la columna payload de transacciones_limpias; no todos aparecen como columnas SQL independientes.

3. Fechas, cuentas e identidad

Fechas que se puedan interpretar

fecha queda en formato **YYYY-MM-DD**. fecha_hora conserva hora y zona. Los ATM sin zona reciben UTC−06:00 según la regla del sistema. Para digital se aceptan segundos Unix o una fecha ISO con zona; una fecha digital sin zona se rechaza.

Una fecha como 03/04/2026 puede significar dos cosas. El sistema no adivina ese formato ambiguo. Estandarizar no debe inventar información que no conocemos.

Cruce con el catálogo

Se busca id_cuenta entre las cuentas del catálogo. Si existe, se agregan sus atributos. Si no existe, la fila va a cuarentena con CUENTA_NO_REGISTRADA. La versión del catálogo queda asociada a la ejecución para saber qué referencia se usó.

Ejemplo real	Resultado
ATM-TX-00009 / CTA-8997	Cuenta ausente: cuarentena en fase INTEGRACION.
ATM-TX-00034 / CTA-0021	Cuenta existente: continúa el proceso. Su estado BLOQUEADA se conserva como atributo.

En este cruce se verifica existencia. No estamos autorizando un retiro real ni bloqueando operaciones bancarias en tiempo real. Por eso una cuenta existente con estado BLOQUEADA puede aparecer en datos aceptados.

Evitar duplicar o contradecir operaciones

La identidad combina fuente e ID. Una huella resume el contenido normalizado de origen. Mismo ID y mismo contenido permite reconocer una repetición; mismo ID y distinto contenido es un conflicto. La representación de origen conserva faltantes para que una estimación posterior no cambie la identidad.

> El catálogo tiene 50 cuentas. Se reportan 56 incidencias de cuenta huérfana entre las rechazadas. Los motivos de rechazo pueden solaparse y algunos controles ocurren antes que otros; no sumes motivos como si fueran filas diferentes.

4. MinMax y segmentación

Después de IQR, se obtiene una variable entre 0 y 1. Esto ayuda a comparar magnitudes dentro del mismo grupo. Se calcula por fuente y operación, sobre monto_analitico_mxn.

```
normalizado = (valor - mínimo) / (máximo - mínimo)
```

Ejemplo: ATM-TX-00034

Monto analítico: **4,000**. En PAGO_SERVICIOS el mínimo es **500** y el máximo **12,000**. Entonces (4000 − 500) / (12000 − 500) = **0.3043478261**. No significa 30 % de probabilidad ni 30 % de riesgo.

Los cortes de segmentación de ese grupo son **1,500** y **8,000**. Hasta el primer corte es BAJO; por encima del primero y hasta el segundo es MEDIO; por encima del segundo es ALTO. El ejemplo de 4,000 queda MEDIO.

Regla	Qué debes explicar
Cortes por cuantiles	Percentiles 1/3 y 2/3; se calculan con interpolación.
Valores repetidos	Pueden producir grupos con cantidades diferentes.
Mínimo igual a máximo	La normalización es 0; se evita dividir entre cero.
Consulta de saldo	Normalizado 0 y segmento BAJO por regla especial.
Parámetros por lote	Cada ejecución conserva sus rangos y cortes.

> No compares dos valores normalizados de lotes diferentes sin revisar sus parámetros. El 1.0 significa llegar al extremo superior de la escala usada, no ser la operación más alta de toda la historia.

En la base entregable hay 452 registros BAJO, 338 MEDIO y 282 ALTO. Total: 1,072. Segmento es una categoría de magnitud; no es una clasificación de fraude.

5. Encontrar el resultado

Práctica desde la raíz del repositorio

```
.venv\Scripts\python.exe entrega/guias/consultar_ejemplos.py avilix
```

Abre una terminal en la carpeta Pipeline_Banco y ejecuta esa línea. El consultor muestra los dos registros reales y abre la base entregable en modo de solo lectura. Necesitas el entorno .venv del proyecto.

Ejercicio: localiza original_json, monto_original, moneda_original, monto_mxn y monto_analitico_mxn. Explica cada diferencia. Después busca parámetros de escala y reproduce el 0.3043478261 del pago de servicios.

Si usas un visor de SQLite

Abre entrega/pipeline_banco.sqlite y ejecuta la consulta. json_extract lee un campo dentro del documento JSON guardado en payload.

```
SELECT id_transaccion,
 json_extract(payload, '$.moneda_original') AS moneda,
 json_extract(payload, '$.monto_original') AS original,
 json_extract(payload, '$.monto_mxn') AS mxn,
 json_extract(payload, '$.monto_analitico_mxn') AS analitico
FROM transacciones_limpias
WHERE id_transaccion = 'SPEI-2026-00003';
```

Qué recibes y qué entregas

De Maki necesitas funciones de calidad con sus parámetros. De Leo necesitas los modelos, errores y persistencia. Tú entregas a persistencia registros canónicos, resultados analíticos y evidencia del lote. El contrato compartido en docs/CONTRATO_DATOS.md evita que cada módulo invente nombres incompatibles.

> La base oficial es entrega/pipeline_banco.sqlite. outputs/pipeline.sqlite puede contener pruebas o demostraciones. Para explicar cifras de entrega, usa siempre la primera.

Cómo explicarlo con tus palabras

Una explicación breve

“Mi parte une las fuentes, estandariza fechas y nombres, convierte USD a MXN con tasa fija 19.50 y cruza cuentas con el catálogo. Después de la limpieza aplico MinMax y segmentos. Conecto las etapas y conservo originales y parámetros para que cada transformación sea explicable.”

Preguntas que debes poder responder

¿Convertimos todo a MXN?
Creamos el importe comparable en MXN y conservamos el importe y la moneda originales.

¿Por qué existe monto analítico?
Para limitar la influencia estadística de extremos sin alterar el importe monetario.

¿Qué es una cuenta huérfana?
Una referencia de cuenta que no existe en el catálogo usado.

¿MinMax detecta fraude?
No. Escala una magnitud dentro del grupo y sus parámetros.

Comprueba que lo entiendes

Explica un ejemplo sin leer, señala la función que lo realiza y encuentra su resultado en la base. Distingue un dato observado, una estimación y una regla de negocio. Si puedes ejecutar el ejemplo y justificarlo, ya tienes una base para defenderlo.

Fuentes de esta guía

Guía del profesor: GUIA_LINEAMIENTOS_PROYECTO_ALUMNOS.pdf, puntos 4 y 6, páginas 2 y 3. Código: src/lectura.py, src/integracion.py, src/transformacion.py, src/pipeline.py, docs/CONTRATO_DATOS.md. Evidencia: entrega/pipeline_banco.sqlite, entrega/resultados.json y entrega/pruebas.txt. Las cifras corresponden a la base entregable original.